

# VIT BHOPAL UNIVERSITY

## PROJECT REPORT

# BADMINTON SPORTS MANAGEMENT SYSTEM

*A Python-based Project for Introduction to Problem Solving and Programming*

|                            |                                                       |
|----------------------------|-------------------------------------------------------|
| <b>Submitted By</b>        | Harsh Vardhan Pal                                     |
| <b>Registration Number</b> | 26BEC10077                                            |
| <b>Programme / Branch</b>  | B.Tech – Electronics and Communication<br>Engineering |
| <b>Course</b>              | Introduction to Problem Solving and Programming       |
| <b>Faculty</b>             | Adarsh Patel                                          |
| <b>Academic Year</b>       | 2026–27                                               |

**VIT Bhopal University**

## **TABLE OF CONTENTS**

1. Introduction
2. Problem Statement
3. Objectives
4. Scope of the Project
5. System Design
6. Modules and Features
7. Algorithms
8. Data Structures Used
9. Program Flow
10. Testing
11. Limitations and Future Scope
12. Conclusion

## 1. INTRODUCTION

Managing players and badminton matches manually can become inconvenient when the number of players increases. Basic details such as player identity, department, scheduled matches, match scores and results need to be organized properly. This project provides a simple console-based solution for these tasks.

The program is written in Python and follows a menu-driven approach. The application is divided into Player Management and Match Management. Player Management handles registration, display and searching of players. Match Management handles scheduling, displaying, searching and recording results of matches.

## 2. PROBLEM STATEMENT

To develop a simple Python-based badminton management system that can store player details, schedule matches between registered players, record the scores of games, determine the winner, and maintain basic player statistics.

## 3. OBJECTIVES

- To create a simple menu-driven application using Python.
- To store and manage badminton player information.
- To schedule matches between two registered players.
- To record scores for up to three games in a match.
- To determine the winner based on games won.
- To maintain basic player statistics such as matches played, wins and losses.
- To provide search options for both players and matches.
- To apply fundamental programming concepts learned in the course.

## 4. SCOPE OF THE PROJECT

The current version is designed for small-scale academic or college-level use. It focuses on individual player records and one-on-one badminton matches. It does not include a database, graphical user interface, online connectivity, authentication, tournament brackets, or automatic score validation. The system is therefore suitable as a basic first-semester programming project.

## 5. SYSTEM DESIGN

The system starts with a main menu. From the main menu, the user can select Player Management, Match Management, or Exit. Player Management contains options to add, display and search players. Match Management contains options to schedule a match, display all matches, record a result and search for a match.

| Component | Purpose                              | Main Python Concept  |
|-----------|--------------------------------------|----------------------|
| players   | Stores player details and statistics | Dictionary           |
| matches   | Stores scheduled/completed matches   | List of dictionaries |

|               |                                    |                         |
|---------------|------------------------------------|-------------------------|
| match_counter | Generates match IDs                | Integer variable        |
| Player Menu   | Controls player-related operations | Function + loop         |
| Match Menu    | Controls match-related operations  | Function + loop         |
| Main Menu     | Controls the complete application  | While loop + conditions |

## 6. MODULES AND FEATURES

### 6.1 Player Management

The player module stores the player ID, name, gender, department, matches played, wins, losses and points. The functions used are add\_player(), display\_players() and search\_player().

### 6.2 Match Management

The match module allows two registered players to be selected for a match. The user enters the date and time, after which a unique match ID is assigned. Match records contain the participating players, schedule, status, scores and winner.

### 6.3 Result Recording

The record\_match() function accepts scores for Game 1 and Game 2. If both players have won one game each, a third game is recorded. The player who wins more games is declared the winner. The match status is changed to completed and player statistics are updated.

### 6.4 Search Functions

The application provides search\_player() for locating a player using the player ID and search\_match() for locating a match using its match ID.

## 7. ALGORITHMS

### 7.1 Algorithm – Add Player

1. Start.
2. Ask the user to enter a player ID.
3. Check whether the player ID already exists.
4. If it exists, display an error and return to the menu.
5. Otherwise, accept name, gender and department.
6. Create a player record with initial statistics set to zero.
7. Store the record in the players dictionary.
8. Display successful registration message.

9. Stop.

## **7.2 Algorithm – Schedule Match**

1. Start.

2. Check whether at least two players are registered.

3. Accept the IDs of Player 1 and Player 2.

4. Verify that both IDs exist and are different.

5. Accept match date and time.

6. Create a match record with status 'scheduled'.

7. Store the record in the matches list.

8. Display the match details and match ID.

9. Increment the match counter.

10. Stop.

## **7.3 Algorithm – Record Match Result**

1. Start.

2. Check whether any matches are scheduled.

3. Accept the match ID and locate the selected match.

4. Read the scores of Game 1 and Game 2.

5. Count which player won each game.

6. If both players have one game win, record Game 3.

7. Compare the total games won by both players.

8. Declare the player with more game wins as the winner.

9. Store the game scores and winner in the match record.

10. Change match status to completed.

11. Increase matches played and update wins/losses.

12. Award the winner's points.

13. Display the final result.

14. Stop.

## 8. DATA STRUCTURES USED

The project mainly uses dictionaries and lists. The players dictionary uses the player ID as a key and stores the player's information as another dictionary. The matches list stores each match as a dictionary.

| Data Structure         | Example                   | Use                           |
|------------------------|---------------------------|-------------------------------|
| Dictionary             | players[player_id]        | Stores player information     |
| List                   | matches                   | Stores multiple match records |
| Nested dictionary      | players[player_id]["won"] | Stores individual statistics  |
| List inside dictionary | score1 / score2           | Stores game scores            |

## 9. PROGRAM FLOW

The overall flow of the program is as follows:

- Start the program.
- Display the main menu.
- Select Player Management or Match Management.
- Perform the selected operation.
- Return to the relevant menu after completing an operation.
- Return to the main menu when the user chooses the back option.
- Exit the program when option 3 is selected.

Main flow representation: START → MAIN MENU → PLAYER MANAGEMENT / MATCH MANAGEMENT → PERFORM OPERATION → RETURN TO MENU → EXIT.

## 10. TESTING

The following test cases can be used to check the intended behavior of the application.

| Test Case | Input / Action                    | Expected Result                         | Status       |
|-----------|-----------------------------------|-----------------------------------------|--------------|
| TC01      | Add a new player                  | Player record is created                | To be tested |
| TC02      | Add an existing player ID         | Duplicate ID message is displayed       | To be tested |
| TC03      | Display players                   | Registered player details are displayed | To be tested |
| TC04      | Search valid player ID            | Player information is displayed         | To be tested |
| TC05      | Schedule match with two valid IDs | Match is stored with unique ID          | To be tested |

|      |                                    |                                           |              |
|------|------------------------------------|-------------------------------------------|--------------|
| TC06 | Schedule match with same player ID | Operation is rejected                     | To be tested |
| TC07 | Display matches                    | Scheduled/completed matches are displayed | To be tested |
| TC08 | Record a match result              | Scores and winner are stored              | To be tested |
| TC09 | Search valid match ID              | Match details are displayed               | To be tested |
| TC10 | Exit                               | Program terminates                        | To be tested |

## 11. LIMITATIONS AND FUTURE SCOPE

### 11.1 Current Limitations

- Data is stored only while the program is running; there is no permanent database or file storage.
- The interface is console-based.
- The current project focuses on singles-style player-versus-player match records.
- There is no login or user authentication system.
- There is no graphical representation of statistics.
- Score validation and advanced tournament management are not included in the present version.

### 11.2 Future Scope

- Add file handling or a database such as SQLite for permanent storage.
- Add a graphical user interface using Tkinter.
- Add tournament creation and knockout/league scheduling.
- Add doubles/team management.
- Generate player rankings and performance reports.
- Add automatic badminton score-rule validation.
- Export match and player statistics to CSV or Excel.

***#Below is the output or the final result being produced after writing the whole source code for python project on badminton sports management system:--***

```
PS C:\Users\hp\AppData\Local\Programs\Microsoft VS Code> & C:\Users\hp\AppData\Local\Programs\Python\Python311\python.exe "c:/Users/hp/downloads/PYTHON PROJECT.py"
c:/Users/hp/downloads/PYTHON PROJECT.py:183: SyntaxWarning: "is" with a literal. Did you mean "=="?
  if selected_match is "none":

=====
BADMINTON SPORTS MANAGEMENT
=====
1. player management
2. match management
3. exit
enter your choice:1

===PLAYER MANAGEMENT===
1. add player
2. display players
3. search player
4. back to main menu
enter your choice :1

=====add player=====
enter player id:sonakshi
enter player name:sonakshi dhiman
enter gender(M/F):F
enter the branch:arts
player registered successfully

===PLAYER MANAGEMENT===
1. add player
2. display players
3. search player
4. back to main menu
enter your choice :2

=====registered players=====
/n=====
player id: sonakshi
name: sonakshi dhiman
gender: F
department: arts
matches: 0
wins: 0
losses: 0
```

```
PS C:\Users\hp\AppData\Local\Programs\Microsoft VS Code> & C:\Users\hp\AppData\Local\Programs\Python\Python311\python.exe "c:/Users/hp/downloads/PYTHON PROJECT.py"
c:/Users/hp/downloads/PYTHON PROJECT.py:183: SyntaxWarning: "is" with a literal. Did you mean "=="?
  if selected_match is "none":

=====
BADMINTON SPORTS MANAGEMENT
=====
1. player management
2. match management
3. exit
enter your choice:2

===MATCH MANAGEMENT===
1. schedule match
2. display all matches
3. record match result
4. search match
5. back to main menu
enter your choice:1
/n===schedule match===
atleast two players are required.

===MATCH MANAGEMENT===
1. schedule match
2. display all matches
3. record match result
4. search match
5. back to main menu
enter your choice:3
/n===record match result===
no matches scheduled.

===MATCH MANAGEMENT===
1. schedule match
2. display all matches
3. record match result
4. search match
5. back to main menu
enter your choice:
```

```
File Edit Selection View Go Run Terminal Help
Python 3.11 (64-bit)

PS C:\Users\hp\AppData\Local\Programs\Microsoft VS Code> & C:\Users\hp\AppData\Local\Programs\Python\Python311\python.exe "c:/Users/hp/downloads/PYTHON_PROJECT.py"
c:/Users/hp/downloads/PYTHON_PROJECT.py:183: SyntaxWarning: "is" with a literal. Did you mean "=="?
  if selected_match is "none":

=====
BADMINTON SPORTS MANAGEMENT
=====
1. player management
2. match management
3. exit
enter your choice:1

===PLAYER MANAGEMENT===
1. add player
2. display players
3. search player
4. back to main menu
enter your choice :3

player found
enter player id:harsh
player not found

===PLAYER MANAGEMENT===
1. add player
2. display players
3. search player
4. back to main menu
enter your choice :3

player found
enter player id:sonakshi
player not found

===PLAYER MANAGEMENT===
1. add player
2. display players
3. search player
4. back to main menu
enter your choice :1
```

```
File Edit Selection View Go Run Terminal Help
Python 3.11 (64-bit)

C:\> Users > hp > Downloads > PYTHON PROJECT.py
339 def match_menu():
340     choice = input("enter your choice:")
341     if choice == "1":
342         schedule_match()
343     elif choice == "2":
344         display_matches()
345     elif choice == "3":
346         record_match()
347     elif choice == "4":
348         search_match()
349     elif choice == "5":
350         break
351     else:
352         print("invalid choice")
353
354 # =====
355 # MAIN MENU
356 # =====
357 while True:
358     print("\n=====")
359     print(" BADMINTON SPORTS MANAGEMENT ")
360     print("=====")
361     print("1. player management")
362     print("2. match management")
363     print("3. exit")
364     choice = input("enter your choice:")
365     if choice == "1":
366         player_menu()
367     elif choice == "2":
368         match_menu()
369     elif choice == "3":
370         print("thank u for using")
371         break
372     else:
373         print("invalid choice")
374
375 # =====
376 # THANK YOU
377 # =====
```

## 12. CONCLUSION

The Badminton Sports Management System demonstrates how basic Python programming concepts can be combined to create a practical console application. The project organizes player records and match information and provides separate menus for managing them. Through this project, concepts such as functions, dictionaries, lists, loops, conditional statements, input/output, and modular program structure can be applied to a real-world style problem. The system can be further improved by adding permanent data storage, a graphical interface, tournament features and more advanced statistics.

### APPENDIX A – PROGRAM MODULE SUMMARY

| Function / Component | Purpose                     | Inputs                       |
|----------------------|-----------------------------|------------------------------|
| add_player()         | Registers a new player      | ID, name, gender, department |
| display_players()    | Displays registered players | None                         |
| search_player()      | Finds a player              | Player ID                    |
| schedule_match()     | Schedules a match           | Two player IDs, date, time   |
| display_matches()    | Displays match records      | None                         |
| record_match()       | Records scores and result   | Match ID, game scores        |
| search_match()       | Finds a match               | Match ID                     |
| player_menu()        | Controls player operations  | Menu choice                  |
| match_menu()         | Controls match operations   | Menu choice                  |
| Main menu            | Controls the application    | Menu choice                  |

### APPENDIX B – NOTE ON IMPLEMENTATION

This report describes the intended design and functionality represented by the submitted source code. Before final submission, the program should be run end-to-end and any syntax or variable-name errors should be corrected. In particular, the submitted source contains several implementation typos in the result-recording and match-search sections. These should be fixed and retested before the final demonstration.

***THANK YOU***